

An Agent-Oriented Framework to Support Rapid Early-Stage Object-Oriented Design

Yilong Yang^{1*}, Runkun Zhang¹, Zhen Tian¹, Weiru Wang²

¹School of Software, Beihang University, Beijing, 100191, China.

²College of Computer Science, Beijing University of Technology, Beijing, 100083, China.

*Corresponding author(s). E-mail(s): yilongyang@buaa.edu.cn;
Contributing authors: runkunzhang@buaa.edu.cn;
zhentian1@buaa.edu.cn; weiru.wang@bjut.edu.cn;

Abstract

Object-Oriented (OO) design, as a cornerstone methodology in software engineering, has been playing a pivotal role in software development, during which OO design models are often crafted. However, the challenge is always on intensive cognitive effort and expertise required for modeling. In the past, automatically generating models from requirements has been considered a promising direction to overcome this challenge. The model-driven engineering (MDE) community has intensively made effort towards this direction. Though some progresses have been made with the help of natural language processing (NLP) techniques, fully automated solutions are rare due to the limitation of NLP techniques themselves, and generated models are not full correct and/or complete. Towards this direction, in this paper, we introduce an agent-oriented methodology with tool support, named RapidOOD, which leverages large language models (LLMs) for the generation, quality assessment, and refactoring of OO design models, by taking requirements models as the input. Our evaluation of RapidOOD with six requirement models demonstrates that automatically generated and subsequently optimized design models achieve 89.20% (without human) and 94.56% (human-in-the-loop collaboration) of expert-level quality metrics, closely matching models crafted manually by experts. Furthermore, our approach achieves 1.39 times the efficiency of the manual method.

Keywords: Object-Oriented, Agent-Oriented, Model Generation, Model Refactoring, Large Language Model

1 Introduction

Software design is a fundamental part of software development, covering two primary stages: architectural design and detailed design[1]. The object-oriented design method, which facilitates the creation of different abstract types during the design process, is more intuitive than the process-oriented design method and has garnered considerable attention. After decades of evolution, object-oriented design has become one of the main approaches to software design and development[2]. Utilizing models is a vital supplementary technique in software design. Since textual descriptions are challenging to employ for development, review, and maintenance, abstraction through graphical modeling languages is used to enhance communication between customers and developers, support requirements validation, facilitate architectural and detailed design, aid in prototyping, enable code generation, and more. With Unified Modeling Language(UML), requirements can be modeled as a lightweight formal requirements model that consists of a use case diagram, a conceptual class diagram, use case definitions specified by system sequence diagrams, and the contracts of their system operations[3–5]. A system operation contract is formally specified by a pair of pre- and post-conditions in Object Constraint Language (OCL). Designs can be modeled using class diagrams and sequence diagrams. Works in requirements phase like RM2PT[6–9] provides features for requirements modeling, analysis, and automated prototype generation from requirements models. Users can rapidly validate and evolve functional requirements through RM2PT to produce high-quality validated requirements models. The validated requirements model can be used as input to the design phase.

Object-oriented design in software development is a knowledge and effort-intensive activity. Designers must invest significant time and effort to create an appropriate design solution for given requirements. Different scenarios and requirements necessitate different design solutions and trade-offs, often depending on the designer’s experience and subjective judgment. Additionally, evaluating the design quality early in the project is challenging and usually not feasible.

This paper aims to reduce the time and effort required during the software design phase through automated methods, lessen the dependence on the designer’s expertise, and provide a quantifiable and comprehensible way to reflect design quality early in the project. The main difficulties and challenges in generating design from requirements and subsequently evaluating and optimizing the initial design include:

1. **Requirements capture stakeholders’ needs, goals, and desired system behaviors. In our context, we focus on functional requirements specified through lightweight formal models,** while design describes system’s internal implementation. Design is a concretization and refinement of requirements, containing more detailed information. It is challenging to supplement the missing information needed to transition from requirements to design.
2. Contracts describe the system state constraints before and after executing operations. Key challenges include determining whether there is a mapping from system states to system implementations (combinations of OO basic operations) and how to map these into operation sequences automatically.

75 3. In practice, design often requires multiple iterations of quality assessment and refac-
76 toring. A critical challenge is adopting suitable iteration strategies and methods to
77 ensure iterations progress toward improved design.

78 These challenges highlight the complexity of software design, underscoring the
79 potential benefits of automating certain aspects of the design process to improve
80 efficiency, quality, and adaptability to changing technological trends.

81 This paper presents an agent-oriented methodology named RapidOOD. It supports
82 the rapid generation, quality assessment, and refactoring of object-oriented designs.
83 This methodology integrates model-driven rule-based agents and LLMs Agents.
84 RapidOOD takes lightweight formal requirement models[10] as input, automatically
85 generates initial object-oriented design models, and computes design quality metrics
86 based on the model elements. It can offer simple rule-based refactoring suggestions
87 based on these quality metrics, and it is also capable of providing complex refactoring
88 recommendations based on LLMs. Designers can iterate on the design model regarding
89 the quality metrics and refactoring suggestions, where each modification’s impact on
90 design quality is immediately reflected through the metrics. A relatively high-quality
91 design can be quickly achieved by automating generation, quality assessment, and
92 refactoring suggestions.

93 We have developed a tool to automate the method. This method was evaluated
94 using six requirement models. In terms of generating design models, the main focus
95 was on assessing the coverage of the proposed rules, with the quality assessment results
96 represented by the ratio of successfully generated class diagram elements and sequence
97 diagram elements. The quality of the automatically generated design models and the
98 quality of the design models after a few iterations were evaluated against the quality
99 of models manually constructed by experts. The time savings achieved by adopting
100 this method were assessed on the six cases. Overall, the experimental results were
101 satisfactory.

102 The contributions of the paper are as follows:

- 103 1. **An Agent-Oriented Approach that Integrates Model-Driven Rule-Based**
104 **Agents and LLM Agents:** This paper presents an agent-oriented methodology
105 which facilitates the rapid generation, assessment, and refactoring of object-oriented
106 designs. This methodology combines model-driven rule-based agents and LLM
107 agents. The software design process is partitioned into an initial design phase fea-
108 turing low degrees of freedom, an quality assessment phase, and an refactoring
109 phase with high degrees of freedom. During the phases with low degrees of freedom,
110 rule-based Agents are utilized to avoid problems like hallucinations resulting from
111 the premature application of LLMs. In the phases with high degrees of freedom,
112 LLMs are incorporated to fully exploit their knowledge-emergence capabilities.
- 113 2. **A Method for Analyzing the Executability of OCL Contracts and Path**
114 **Reasoning:** Our work introduces an automated approach for analyzing the exe-
115 cutability of a given OCL contract and determine a reasonable implementation
116 path. This method effectively bridges the gap between requirements and design,
117 providing an efficient, automated pathway for object-oriented software design.

118 3. Design refactoring with Embedding quality assessment Results in LLM

119 **Prompts:** Utilizing a model-driven methodology, this research advances design
120 quality quality assessment from code-based assessments to an innovative model-
121 based framework. The design quality quality assessment results obtained through
122 the model-driven method are embedded into LLM Prompts to guide the LLM to
123 propose corresponding refactoring suggestions

124 The rest of this article is organized as follows. Section 2 discusses the related work.
125 Section 3 presents the background of our approach. Section 4 introduces RapidOOD
126 in detail, including how to automatically generate design models from requirements
127 models, how to evaluate design quality, and how to give refactoring suggestions. Section
128 5 presents the evaluation of our approach. Section 6 concludes this article.

129 2 Related Work

130 According to the existing research, the automatic generation of software design mod-
131 els can be divided into two types according to the input and generation methods. In
132 some research, the input is mainly the requirement described by natural language.
133 The requirement description is processed using the related natural language processing
134 methods to generate the design model. In other research, the input is the requirement
135 model, which uses the model-driven method to convert requirement models to design
136 models by defining the transformation rules. Each study presents requirements and
137 design models differently but contains consistent information. Requirement models
138 generally describe what entities exist in the system, the relationships between them,
139 what services the system can provide to users, what operations the system can com-
140 plete, and the system’s state before and after operations, which information users can
141 obtain from outside the system. Design models generally describe information related
142 to system implementation, such as how classes within the system are organized, how
143 classes relate to each other, and how system operations are accomplished by instances
144 of classes in collaboration.

145 2.1 Design Generation from Requirements Description

146 In terms of generating design models from requirements described in natural language,
147 Mala et al.[11] proposed a method to extract object-oriented elements from natural
148 language. This method divides the original requirements into sentences, labels each
149 word in the sentence, and then recombines nouns and verbs through a combiner based
150 on simple phrase grammar to obtain a relatively structured requirements directory.
151 Then, words with different parts of speech are translated into classes, attributes, and
152 use cases through the defined conversion rules. For example, nouns can be translated
153 into classes and attributes of classes. The disadvantage of this work is that the identi-
154 fied classes may be more than the actual classes. The generated class does not contain
155 methods, cannot describe the interaction between objects within the system, and does
156 not contain the information corresponding to the sequence diagram.

To reduce the ambiguity caused by the rich semantics of the natural language, V. s. sharma[12] and others proposed a method to extract functional designs from relatively structured restricted natural languages and generate UML class diagrams and implemented this method as a tool FDCT (functional design creation tool), which was integrated into IBM Rational Software Architect as a plug-in. The original requirement text is processed by the tool rat produced by its previous work to obtain specific types of strings. The heuristic rules proposed by it are applied to these strings, and finally, a class diagram containing classes, methods and relationships between classes is obtained. Its original requirement text is limited by user-defined vocabulary, and the definition of vocabulary is complex, which is not easy compared with requirement modeling. Only some types of requirements are supported. The generated class diagram does not contain attributes. The internal behavior of the system was not explicitly designed.

Approach *aToucan*[13] automatically generates analysis models consisting of class, sequence, and activity diagrams from requirement models containing natural language. Although *aToucan* can generate sequence diagrams and activity diagrams, their content is limited to human-system interactions and lacks designs to collaborate objects within the system.

2.2 Design Generation from Requirements Model

The most related research to this study is related work on automatically generating design models from lightweight formal requirements models. Steve et al. [14] introduced the lightweight application of formal methods in requirements modeling of spacecraft fault protection systems through three case studies and explained the benefits of introducing lightweight formalization in the requirements stage. Xiaoshan Li et al. [15] proposed a formal method for requirements modeling using UML. This method was further developed by ZhiMing Liu et al.[16–18], who proposed the Refinement of Component and Object Systems (rCOS) method. This paper provides a formal model definition for the UML-based Rational Unified Process (RUP) development process, which can facilitate the introduction of model-driven methods in the requirements, design, and verification phases of component-based object-oriented systems. And better guarantee consistency and correctness, and also provide a basis for introducing automated methods. The manual process from requirements model to a detailed functional design model is introduced in his work. In terms of design model generation from the formal requirements model, Nora koch[19] uses the model transformation method to transform the requirement model into the design model for the web system. The meta-model of the input requirement model is the Webre meta-model derived from summarizing the concepts that need to be used in web system modeling. This meta-model refines the UML meta-model. UWE meta-model defines the output design model. There is a simple mapping relationship between elements in the UWE meta-model and classes and the relationship between classes. Web system design is described in presentation, navigation, and content. The conversion rules of the two models are defined in QVT language, which can realize bidirectional conversion and comply with OMG standards. This research ultimately realizes the transformation

from the web system requirement model to the design model and proves the feasibility of automatically generating the design model based on the model transformation. However, some things could be improved: this research is mainly aimed at the GUI model and does not involve the system operation encapsulation and the interaction of internal objects of the system. G. Mussbacher et al.[20] proposed a transformation method of an oriented model for a reactive system. From the scenario-based requirement model aoucm (aspect-oriented use case maps) to the UML-based design model RAM (reusable aspect models), it further confirms the feasibility of transforming the requirement model into the design model on different systems.

2.3 OO Design Quality Metrics and Quality Optimization

There are four groups of widely accepted design metrics (CK[21] LK[22], MOOD[23], and QMOOD[24]) for software object-oriented design quality metrics and assessment. The latest QMOOD is an improved hierarchical model for evaluating high-level design quality attributes in object-oriented design. The model relates low-level statistical metrics of design with design properties (such as encapsulation, modularity, coupling, and cohesion) and high-quality properties (such as reusability, flexibility, and complexity). Pham[25] et al. used the above four groups of design metrics to propose a set of unified metrics covering many aspects of object-oriented software design. Outside of the set of quality metrics, there is ongoing exploration into different aspects of object-oriented design. In the research of design quality assessment using design metrics, the existing problem is that there are empirical values when the lower-level quality metrics or features are integrated into the upper-level quality attributes, and these empirical values cannot be automatically adjusted according to the system type and domain knowledge. Meng Yan[26] et al. conducted a systematic mapping study on the related research of quality assessment model, and an important conclusion was that there was a lack of tool support in software design assessment.

Regarding software design optimization, there are two paths: quality-driven model transformation and search algorithm. Insfran [27] et al. proposed design guidelines for developing quality-driven model transformations. By using quality models to express quality attributes and transformation models to express alternative model transformations and their effects on quality attributes, transformation designers are allowed to use quality attributes as decision factors for choosing alternative transformations. The correctness of this approach depends on its transformation model and is not flexible enough. The use of search algorithms for software design optimization is mainly the use of genetic algorithms [28–30], including single-objective and multi-objective genetic algorithms. Design optimization schemes proposed using model transformation or search algorithms also fail to incorporate system type and domain knowledge and are less effective.

2.4 Summary of Related Work

- Some existing works provide supplementary information about the system’s internal static implementation (including how classes are organized and what operations they

- 241 have) but fail to include the system’s internal dynamic design information (mainly
 242 how objects within the system interact to complete operations).
- 243 • Existing works do not consider the executability of system operation contracts and
 244 the sequencing of atomic operations.
 - 245 • Existing works do not explain the impact of each iteration on design quality in
 246 an easily understandable way, nor do they integrate the quality assessment and
 247 refactoring phases in an automated manner.
 - 248 • To the best of our knowledge, existing work has not effectively combined model-
 249 driven methods with LLMs for application in software design tasks.

250 2.5 Comparison to RM2DM

251 Our previous tool, RM2PT, facilitated rapid prototyping from requirement models to
 252 swiftly confirm requirements. Building on these confirmed requirements, our previous
 253 work, RM2DM[31], introduces a tool that can swiftly generate design models adhering
 254 to the anemic model design concept from the confirmed requirements. Despite the
 255 anemic model design being considered an anti-pattern, it is extensively employed in
 256 practical project development, such as in Java EE and Spring, which are both based
 257 on the anemic model concept. In comparison to our previous work, the method and
 258 tool presented in this paper offer the following:

- 259 • *Detailed transformation rules from requirements to sequence diagrams in design
 260 models:* Our previous work only provided tool implementation without explicitly
 261 stating the rules for generating sequence diagrams in the design models. In this
 262 paper, we detail the transformation rules and their use cases.
- 263 • *Integration of automatic quality assessment and refactoring methods based on the
 264 generated design models:* Our previous work could only generate a simple initial
 265 design model. In this method, we use QMOOD metrics and some rules to provide
 266 automatic quality assessment and refactoring capabilities, further addressing the
 267 issue of design automation. Corresponding tool support is also provided based on
 268 Sirius.

269 **Table 1 summarizes the key differences between RM2DM and the approach presented**
 270 **in this paper across five dimensions.**

Table 1 Comparison of Key Capabilities between RM2DM and This Work

Dimension	RM2DM	This Work
Class Diagram Generation	✓ Already in RM2DM	✓ Already in RM2DM
Explicit Sequence Diagram Rules	–	New in This Work
Quality Assessment & Refactoring	–	New in This Work
LLM-based Refactoring Suggestions	–	New in This Work
Requirement Traceability	✓ Already in RM2DM	✓ Already in RM2DM

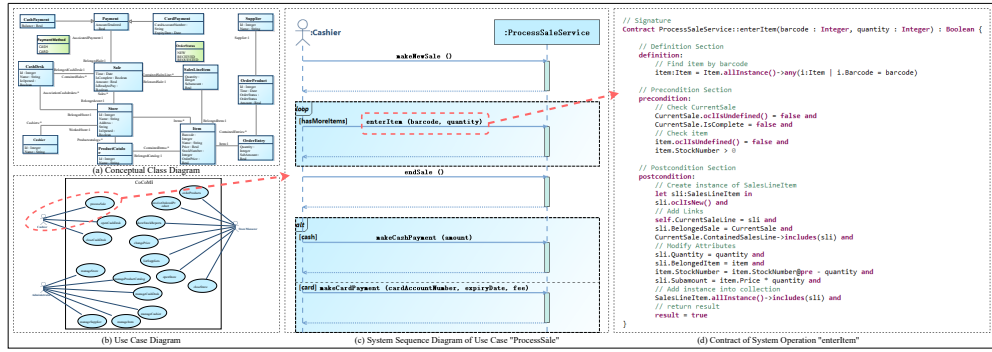


Fig. 1 Requirements Model

3 Preliminary

3.1 Requirements and Design Models

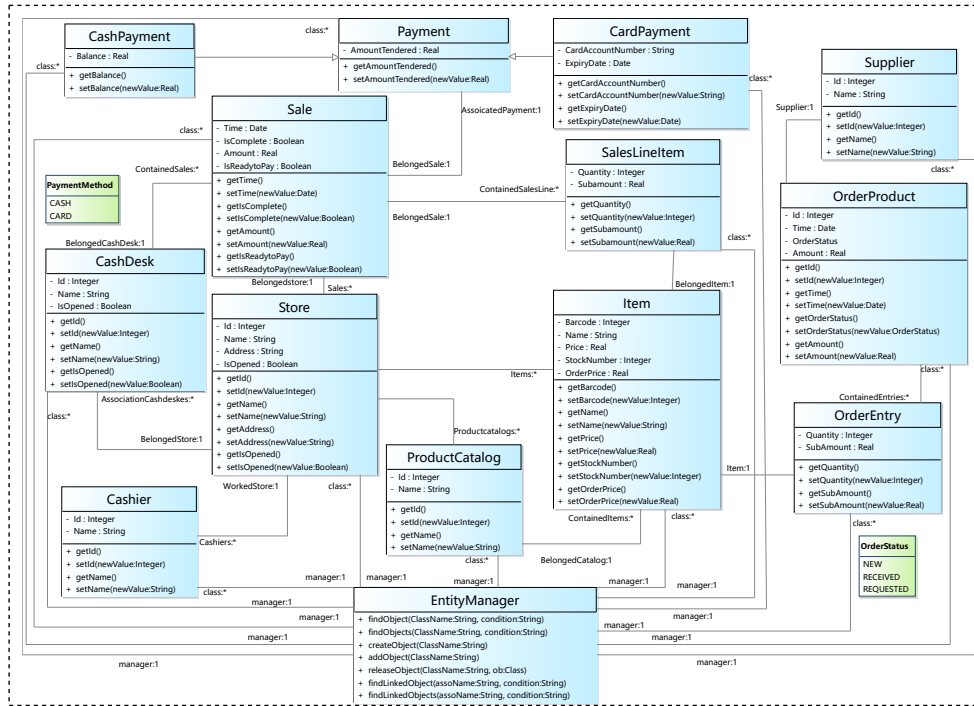
The requirements model adopted in our approach consists of four components, as illustrated in Figure 1:

- **Use Case Diagram:** Describes interactions between actors and the system to capture user goals.
- **Conceptual Class Diagram:** Depicts domain concepts and their relationships, without specifying operations or implementation details.
- **System Sequence Diagrams:** Shows the sequence of system operations for each use case, treating the system as a black box.
- **Contracts of System Operations:** Formally specifies pre- and post-conditions of each system operation using OCL[3–5, 8, 10].

The design model generated by our approach comprises a class diagram and sequence diagrams. The class diagram defines the static structure, including classes, attributes, operations, and relationships such as aggregation, composition, and inheritance. Sequence diagrams capture the dynamic interactions among objects by detailing the messages exchanged to realize each system operation. The key distinction from the requirements model is that design-level class diagrams specify operations and encapsulation strategies, while sequence diagrams reveal internal object interactions rather than treating the system as a black box. Examples of the generated design model are shown in Figures 2, 3, and 4.

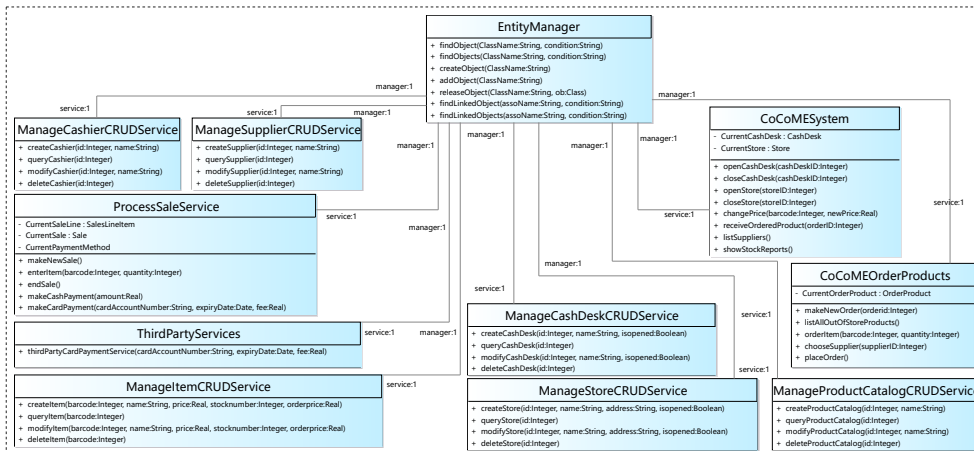
3.2 Anemic Domain Model

The anemic domain model [32] structures the system into three layers: Domain (entities and value objects with data and basic operations), Service (business logic), and Repositories (data persistence and querying). Unlike rich domain models where entities encapsulate both data and business logic, the anemic model places business logic in the Service layer, making entities thin data containers. Despite being considered a



(a) Entity Classes of Generated Class Diagram

Fig. 2 Generated Class Diagram Entity Part



(b) Service Classes of Generated Class Diagram

Fig. 3 Generated Class Diagram Service Part

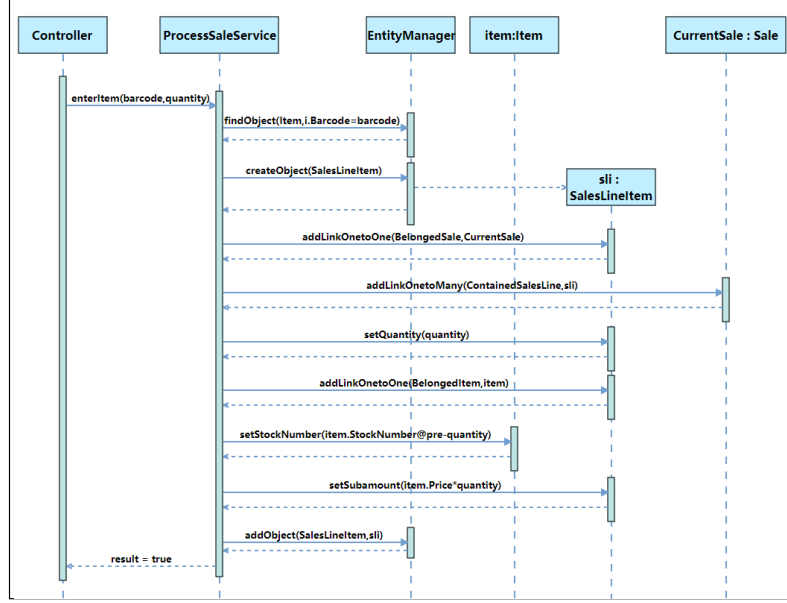


Fig. 4 Generated Sequence Diagram for System Operation “enterItem”

design anti-pattern, the anemic model is widely adopted in practice, particularly in web applications where business logic primarily involves CRUD operations, and is supported by mainstream frameworks such as Java EE and Spring. Our approach targets this prevalent architectural style.

3.3 Rule-based Agents and LLMs-based Agents

Software agents and multi-agent systems have received considerable research attention since the 1980s[33]. An agent, as an autonomous system, is capable of interacting with other agents to fulfill its designed objectives. Multi-Agent Systems (MAS) consist of multiple autonomous entities, or agents, that interact within a shared environment to achieve individual or collective goals. In the research of agent-oriented software engineering, the primary focus is on how to utilize agents as fundamental building modules for designing and developing complex systems, with an emphasis on modularity, reusability, and software architecture. The agents therein typically operate based on predefined rules or logical reasoning, and their interactions mainly base on rule and deterministic protocols or algorithms. We refer to this kind of agent as Rule-based Agents.

Large Language Models are a class of deep learning models designed to understand and generate human-like text. LLMs leverage massive datasets and billions of parameters to achieve remarkable language comprehension and generation capabilities. Due to the rapid development of LLMs, LLMs-based agents have attracted significant attention. LLMs-based agents take LLMs as the core. Through reinforcement learning and function-calling techniques, such agents can automate complex tasks via the “perceive-decide-act” process.

4 Approach

This section describes our method for transforming requirement models into design models (including a class diagram and sequence diagrams), assessing design quality during the design phase, and offering straightforward refactoring recommendations.

4.1 Overview

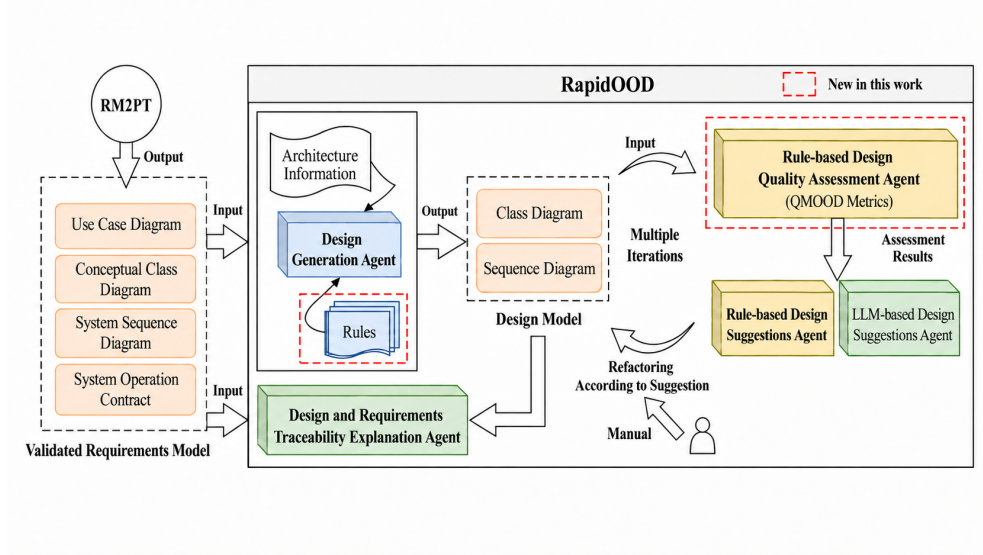


Fig. 5 Overview of RapidOOD. Red dashed boxes indicate capabilities newly introduced in this work compared with RM2DM.

Our approach takes validated requirements models as input. Auto-prototyping work, such as RM2PT, assists in swiftly attaining a validated requirements model by automatically producing prototypes from requirement models, thereby facilitating incremental and agile requirements validation. We propose a set of rules and algorithms to generate the design models composed of a class diagram and a set of sequence diagrams. Based on the initial design models generated, we employ QMOOD, a hierarchical model for evaluating the quality of object-oriented designs, to assess the design quality. The results of the quality assessment metrics and the simple refactoring suggestions we propose are fed back to the users. Users can then manually iterate on the design based on this feedback, thereby swiftly achieving a relatively higher quality design.

In this paper, the agents are not solely centered around LLMs, instead, the framework incorporates a hybrid approach where agents are built upon both LLM-driven capabilities and traditional model-driven methods. While LLM-based agents excel in tasks requiring natural language understanding, reasoning, and adaptability, the

341 model-driven agents leverage predefined rules, domain-specific models, and structured
342 decision-making processes to handle tasks with deterministic logic or stringent perfor-
343 mance requirements. This duality allows the system to balance the flexibility of LLMs
344 with the reliability and efficiency of traditional approaches, creating a more robust
345 and versatile multi-agent framework.

346 In RapidOOD, we designed the Design Generation Agent, Design Quality Assess-
347 ment Agent, and rule-based Design Suggestions Agent that provides simple recom-
348 mendations using a model-driven, rule-based approach. The model-driven approach
349 generates the initial design model from the requirements model, ensuring excellent
350 traceability and the scale of the initial design. (As can be seen in the subsequent
351 experimental section, the design models generated solely by LLMs are limited in scale
352 and often lack essential relationships or operations.) For the quality assessment of the
353 design, a quantitative method based on QMOOD Metrics was adopted. By statistically
354 analyzing the model elements of the design model, high-level quality attribute met-
355 rics of the design model were obtained, such as reusability. For low quality attributes,
356 rule-based design refactoring suggestions were provided. For example, parts with high
357 coupling in the design model were highlighted and suggestions for refactoring were
358 given.

359 We designed the LLM-Based Design Suggestions Agent and the Traceability Expla-
360 nation Agent. The aim is to utilize the knowledge-emergence ability of LLMs to
361 provide expert-level design refactoring suggestions and to offer easily understandable
362 explanations for the traceability between the design and the requirements.

363 4.2 Design Model Generation Agent

364 4.2.1 Transformation Rules of Class Diagram Generation

365 Three transformation rules are proposed to automatically generate a class diagram
366 from the requirement model .

367 *Preserve the Classes and Relationships*

368 The first rule is to preserve the relationships between classes and classes in the concep-
369 tual class diagram and add GET/SET methods corresponding to the class attributes.
370 This design is based on the fact that the relationships between classes and classes in
371 the conceptual class diagram in the requirement model are naturally consistent with
372 the actual situation, and this part should remain the same; otherwise, relationship
373 errors or missing relationships will occur.

374 *Generate Service Class*

375 The second rule is to encapsulate all use case-related system operations into a new Ser-
376 vice class based on the use case diagram (including use cases) and the system sequence
377 diagram (including system operations related to use cases). For a case, according to
378 the information in the use case diagram and the system sequence diagram, a new class
379 is extracted, which contains the class name, attributes, operations, and other infor-
380 mation. It extracts class names from use case diagrams and attributes and operations
381 from system sequence diagrams and system operation contracts.

382 The classes extracted by this rule will serve as the content of the Service layer and
 383 can be used to provide services to the outside world. Service will act as the initiator
 384 of the system operation, and then the various classes within the system will work
 385 together to complete the various system operations.

386 ***Generate EntityManager Class***

387 The third rule involves the creation of a new EntityManager class, which serves as the
 388 central hub for all other classes. It is responsible for adding, deleting, modifying, and
 389 querying objects, and thus should have primitive operations related to these tasks.
 390 The operations owned by EntityManager are derived by summarizing the OCL state-
 391 ments in the system operation contract. 10 types of OCL expressions associated with
 392 EntityManager and 7 primitive operations are needed.

$$\begin{aligned} E_1 : obs : Set(ClassName) &= ClassName.allInstances() \\ E_2 : obs : Set(ClassName) &= ClassName.allInstances() \\ &- > select(o|conditions(o)) \end{aligned}$$

393 The meaning of statement 1 is to find instances of all ClassName classes in the
 394 system, and the meaning of statement 2 is to find instances of conditional conditions
 395 of all ClassName classes in the system. To complete the behavior described by the two
 396 OCL statements, the class EntityManager requires the operation findObjects().

$$\begin{aligned} E_3 : ob : ClassName &= ClassName.allInstances() \\ &- > any(o|conditions(o)) \end{aligned}$$

397 The meaning of statement 3 is to find an instance of a conditional condition of
 398 any ClassName class in the system. Therefore, the class EntityManager requires a
 399 findObject() operation.

$$\begin{aligned} E_4 : o : ClassName &= ob.assoName \\ E_5 : o : ClassName &= ob.assoName \\ &- > any(o|conditions(o)) \end{aligned}$$

400 Statement 4 means to find a single instance of a class in the system that has an
 401 assoName relationship with an ob class, and statement 5 means to find any instance
 402 of a conditional condition of a class that has an assoName relationship with an OB
 403 class in the system. Both statements find a single object based on the association
 404 relationship, so the EntityManager needs an operation findLinkedObject ().

$$\begin{aligned} E_6 : obs : Set(ClassName) &= ob.assoName \\ E_7 : obs : Set(ClassName) &= ob.assoName \\ &- > select(o|conditions(o)) \end{aligned}$$

405 The meaning of statement 6 is to find a collection of instances of a class in the
 406 system that has an assoName relationship with an ob class, and the meaning of state-
 407 ment 7 is to find a collection of instances of a conditional class in the system that has
 408 an assoName relationship with an ob class, so the EnitivityManager needs to be able to
 409 find a collection of objects based on the association relationship and needs to operate
 410 findLinkedObjects ().

$$E_8 : let ob : ClassName in ob.oclIsNew()$$

411 The meaning of statement 8 is to define an object OB for a ClassName, so the
 412 EnitivityManager needs to be able to create instances of a particular class and needs to
 413 have an operation createObject ().

$$E_9 : ClassName.allInstances() -> includes(ob)$$

414 Statement 9 appears in the Postcondition section of the system operation contract,
 415 indicating that the OB object needs to be guaranteed to be included in the instance of
 416 ClassName after the system operation is executed, so EnitivityManager needs to add the
 417 OB object to the list of instances of ClassName class, and the addObject () operation
 418 is required.

$$E_{10} : ClassName.allInstances() -> excludes(ob)$$

419 Statement 10 appears in the Postcondition section of the system operation contract,
 420 indicating that you need to ensure that the ClassName instance after the system
 421 operation is executed does not contain an ob object, so EnitivityManager needs to
 422 remove the OB object from the list of ClassName instances, and you need an operation
 423 removeObject().

424 4.2.2 Transformation Rules of Sequence Diagram Generation

425 RapidOOD uses sequence diagrams to describe the collaboration of objects within the
 426 system. The main content of the sequence diagram is the messages passed between
 427 objects. Based on the anemic domain model, there is a controller in the system. The
 428 controller sends the invocation messages of system operations to the corresponding
 429 service class. The service class sends messages to the related objects inside the system,
 430 and the objects collaborate to complete the system operations and return. The primary
 431 challenge lies in the generation of messages between objects. Of the three components
 432 of the system operation contract, the OCL expressions in the precondition part define
 433 the system's required state before executing the operation. As these preconditions do
 434 not involve interactions between class instances, they are not converted into messages.
 435 The definition and post-condition of the system operation contract, on the other hand,
 436 describe the system's dynamic behavior. The OCL expressions in these sections can be
 437 transformed into messages in sequence diagrams. To facilitate this transformation, a
 438 set of 17 transformation rules has been established. Transformation rules are presented

439 in this form:

$$\text{Rule} : \frac{\text{OCL Expression}}{\text{Message Label, Sender, Receiver}}$$

440 The transformation rule contains two parts: the above is an OCL expression in
441 the contracts, and the bottom part is the generated message and the message's sender
442 and receiver.

$$R1 : \frac{\text{obs: Set (ClassName)} = \text{ClassName.allInstances()}}{\text{findObjects (ClassName), Service, EntityManager}}$$

$$R2 : \frac{\text{obs: Set (ClassName)} = \text{ClassName.allInstances()} \rightarrow \text{select (o | conditions(o))}}{\text{findObjects (ClassName, conditions (o)), Service, EntityManager}}$$

$$R3 : \frac{\text{ob: ClassName} = \text{ClassName.allInstances()} \rightarrow \text{any (o | conditions(o))}}{\text{findObject (ClassName, conditions (o)), Service, EntityManager}}$$

$$R4 : \frac{o: \text{ClassName} = \text{ob.assoName}}{\text{findLinkedObject (ob, assoName), Service, EntityManager}}$$

$$R5 : \frac{o: \text{ClassName} = \text{ob.assoName} - \dot{\wedge} \text{any (o - conditions(o))}}{\text{findLinkedObject (ob, assoName, conditions(o)), Service, EntityManager}}$$

$$R6 : \frac{\text{obs: Set (ClassName)} = \text{ob.assoName}}{\text{findLinkedObjects (ob, assoName), Service, EntityManager}}$$

$$R7 : \frac{\text{obs: Set (ClassName)} = \text{ob.assoName} - \dot{\wedge} \text{select (o - conditions(o))}}{\text{findLinkedObjects (ob, assoName, conditions(o)), Service, EntityManager}}$$

443 Rules 1 to 7 are the conversion rules from the definition part of the OCL statement
444 to the message. The meaning of this part of the statement has been introduced above.
445 According to this part of the conversion rules, the sending end of the message is
446 Service, and the receiving end is EntityManager.

$$R8 : \frac{\text{let ob: ClassName in ob.ocIsNew()}}{\text{createObject (ClassName), EntityManager, ob}}$$

447 The meaning of statement 8 is to generate an instance ob of the ClassName class.
448 The operation of creating an object is initiated by the Service and completed by the
449 EntityManager to generate an object ob. There are actually two request messages in
450 this process. One is that the Service sends a message to the EntityManager to request
451 the creation of an instance; the other is the creation message of the object ob created
452 by the EntityManager.

$$R9 : \frac{\text{ClassName.allInstances()} - \dot{\wedge} \text{includes (ob)}}{\text{addObject (ClassName, ob), Service, EntityManager}}$$

$$R10 : \frac{\text{ClassName.allInstances()} - \dot{\wedge} \text{excludes (ob)}}{\text{releaseObject (ClassName, ob), Service, EntityManager}}$$

453 Statement 9 and statement 10 are the OCL statements of the postcondition part,
454 and the related class is also EntityManager, which has been introduced in the previous
455 article. For the messages obtained according to rules 9 and 10, the sending end is
456 Service, and the receiving end is EntityManager.

$$R11 : \frac{\text{ob.assoName} - \dot{\wedge} \text{includes (addOb)}}{\text{addLinkOnetoMany (assoName, addOb), Service, ob}}$$

457 The meaning of statement 11 is to add the addOb object to the list of objects
 458 pointed to by the assoName relation of ob, corresponding to the atomic operation
 459 addLinkOnetoMany(), which is executed by the object ob.

$$R12: \frac{ob.assoName \text{ - } \exists excludes(removeOb)}{removeLinkOnetoMany(assoName, removeOb), Service, EntityManager}$$

460 The meaning of statement 12 is to remove the removeOb object from the list of
 461 objects pointed to by the assoName relation of ob, and the corresponding atomic
 462 operation is removeLinkOnetoOne(), which is executed by the object ob.

$$R13: \frac{ob.assoName = addOb}{addLinkOnetoOne(assoName, addOb), Service, ob}$$

463 The meaning of statement 13 is to change the object pointed to by the assoName
 464 relation of ob to addOb, and the corresponding atomic operation is addLinkOne-
 465 toOne(), which is executed by the object ob.

$$R14: \frac{ob.assoName = null}{removeLinkOnetoOne(assoName), Service, ob}$$

466 The meaning of statement 14 is to change the object pointed to by the assoName
 467 relation of ob to null, i.e., it does not point to any object, and the corresponding
 468 atomic operation is removeLinkOnetoOne(), which is executed by the object ob.

$$R15: \frac{ob.attriName = mathExp}{setAttriName(mathExp), Service, ob}$$

469 The meaning of statement 15 is to change the value of the attriName attribute of
 470 ob to the value represented by mathExp, and the corresponding atomic operation is
 471 setAttribute(), which is executed by the object ob.

$$R16: \frac{result = var}{return(var), Service, Controller}$$

472 The meaning of statement 16 is to return the result of the system operation.

$$R17: \frac{ThirdPartyService.opName(vars)}{opName(vars), Service, ThirdPartyService}$$

473 The meaning of statement 17 is to call a method of a third-party service. For each
 474 OCL expression in the system operation contract, apply rules 1-7 if it belongs to the
 475 definition part of the contract and rules 8-17 if it belongs to the post-condition part.
 476 Determine the rules for the OCL expression based on its syntactic structure. Apply
 477 the rules to extract the label, the sending object and receiving object of the message
 478 from the OCL expression.

479 Algorithm 1 shows how to apply the rules to generate messages from system opera-
 480 tion contracts. For each OCL expression in the system operation contract, apply rules
 481 1-7 if it belongs to the definition part of the contract and rules 8-17 if it belongs to
 482 the post-condition part. Determine the rules for the OCL expression based on its syn-
 483 tactic structure. Apply the rules to extract the label, the sending object and receiving
 484 object of the message from the OCL expression.

485 Algorithm 2 describes how to order OCL statements to ensure that they are
 486 executable. It involves constructing a dependency graph and performing topological
 487 sorting to ensure that all dependencies are respected. The algorithm details are as
 488 follows.

Algorithm 1 Sequence Diagram Messages Generation

Input: System Operation Contract

Output: Sequence Diagram Messages

```
1: for each OCLExpression  $\in$  Contract do
2:   if OCLExpression  $\in$  definition then
3:     applyRule(1 – 7)
4:   else if OCLExpression  $\in$  postcondition then
5:     applyRule(8 – 17)
6:   end if
7: end for
8: procedure applyRule(ruleOrder)
9:   setMessageLabel()
10:  setMessageSender()
11:  setMessageReceiver()
```

489 **Initialize the Dependency Graph:** Create an empty directed graph $G = (V, E)$,
490 where V represents the set of nodes (OCL statements) and E represents the set of
491 directed edges (dependencies between statements).

492 **Add Nodes to the Graph:** For each OCL statement $s_i \in S$ (where S is the set
493 of OCL statements), add s_i as a node in V .

494 **Add Edges to the Graph:** For each OCL statement $s_i \in S$, identify its depen-
495 dencies. For each dependency s_j of s_i , add a directed edge (s_j, s_i) to E . This indicates
496 that s_i depends on s_j .

497 **Topological Sorting Initialization:** Initialize an empty list L to store the sorted
498 OCL statements. Initialize an empty set S to store all nodes with no incoming edges.

499 **Identify Nodes with No Incoming Edges:** For each node $v \in V$, if v has no
500 incoming edges, add v to S .

501 **Topological Sorting Process:** While S is not empty: - Remove a node n from
502 S . - Add n to L . - For each node m such that $(n, m) \in E$: - Remove edge (n, m) from
503 E . - If m has no other incoming edges, add m to S .

504 **Cycle Detection:** If there are any remaining edges in E , this indicates that the
505 graph G contains at least one cycle, and a topological sorting is not possible. In this
506 case, report an error.

507 **Return the Sorted List:** Return the list L as the topologically sorted order of
508 OCL statements.

Algorithm 2 OCL Statement Execution Order Sorting

Input: A set of OCL statements $S = \{s_1, s_2, \dots, s_n\}$ with dependencies

Output: A topologically sorted list of OCL statements

```
1: Initialize an empty dependency graph  $G = (V, E)$ 
2: for each  $s_i \in S$  do
3:   Add  $s_i$  as a node in  $V$ 
4: end for
5: for each  $s_i \in S$  do
6:   for each  $s_j \in \text{dependencies}(s_i)$  do
7:     Add a directed edge  $(s_j, s_i)$  to  $E$ 
8:   end for
9: end for
10: Initialize an empty list  $L$  to store the sorted OCL statements
11: Initialize an empty queue  $Q$  to store all nodes with no incoming edges
12: for each  $v \in V$  do
13:   if  $v$  has no incoming edges then
14:     Add  $v$  to  $Q$ 
15:   end if
16: end for
17: while  $Q$  is not empty do
18:   Remove a node  $n$  from  $Q$ 
19:   Add  $n$  to  $L$ 
20:   for each node  $m$  such that  $(n, m) \in E$  do
21:     Remove edge  $(n, m)$  from  $E$ 
22:     if  $m$  has no other incoming edges then
23:       Add  $m$  to  $Q$ 
24:     end if
25:   end for
26: end while
27: if any edges remain in  $E$  then
28:   error: Graph  $G$  has at least one cycle
29: end if
30: return  $L$ 
```

4.3 Quality Assessment and Refactoring Recommendations of Design Model

We evaluated design quality using QMOOD, a hierarchical model to evaluate object-oriented design quality. In the QMOOD method, the high-level quality attribute indicators are calculated from the low-level element statistical indicators. Traditionally, this quality assessment is based on code. However, we noticed that many of these underlying metrics were available early in the design phase of the project, empowering designers to make proactive decisions. For example, QMOOD uses the number of

517 classes to represent the size of the design, which is determined when the design is com-
518 pleted. Another example is the number of levels in a hierarchical architecture, whose
519 value is fixed and can also be obtained at design time.

520 4.3.1 Rule-based Design refactoring Agent

521 Based on the results of the quality attributes assessment, we provide simple rule-based
522 design refactoring recommendations to designers. Since the calculation of high-level
523 quality attribute indicators is based on the statistical number of underlying elements,
524 we can identify the design elements that most impact a particular quality attribute
525 indicator. We decided to set the granularity of design elements to classes, and when
526 a class was found to have the most significant negative impact on specific quality
527 attribute metrics, we highlighted it and suggested that designers make refactorings.

- 528 • Design size: Consider the case where the number of classes exceeds a threshold that
529 indicates complexity. This metric is available early in the design phase. If the number
530 of classes is too high, we recommend strategies such as refactoring into smaller
531 modules or applying design patterns like the Factory pattern to manage complexity.
- 532 • Hierarchy: In a hierarchical architecture, if the number of levels is too deep, indi-
533 cating a potential for maintenance issues, we suggest flattening the structure by
534 merging similar classes or using composition over inheritance where appropriate.
- 535 • Coupling: When the coupling between classes is too high, indicating a lack of mod-
536 ularity, we recommend introducing interfaces or applying the Dependency Injection
537 pattern to decouple components.
- 538 • Cohesion: If the cohesion within a class is low, suggesting that the class has multiple
539 responsibilities, we advise splitting the class into smaller, more focused classes or
540 applying the Single Responsibility Principle.
- 541 • Polymorphism: If there is a high degree of polymorphism, indicating a com-
542 plex inheritance hierarchy, we recommend simplifying the inheritance structure by
543 reducing the depth of the hierarchy or employing composition instead of inheritance.

544 For example, in the generated class diagram shown in Figure 2, we can see that the
545 EntityManager class has the most significant number of directly related classes, which
546 has the highest degree of coupling according to the metric calculation, thus suggest-
547 ing a design refactoring for it: “EntityManager has high Direct Class Coupling, please
548 pay attention to proper class responsibility division”. Considering the refactoring of
549 EntityManager, in this design, EntityManager is mainly responsible for receiving the
550 requests of the service layer and modifying the attributes and association relations of
551 the related classes in the entity layer. The EntityManager can be split into multiple
552 repositories, each responsible for modifying a class. This reduces the degree of coupling
553 between classes, and the responsibility of each class is more single, in line with the
554 single responsibility principle. We can see that this modification has changed some of
555 the low-level design metrics and affected the high-level design quality. Understandabil-
556 ity decreases because multiple managers are more complex than a single Manager in
557 terms of design structure. However, splitting it into various managers makes for better
558 readability. Functionality goes up because multiple Managers bring access to multiple

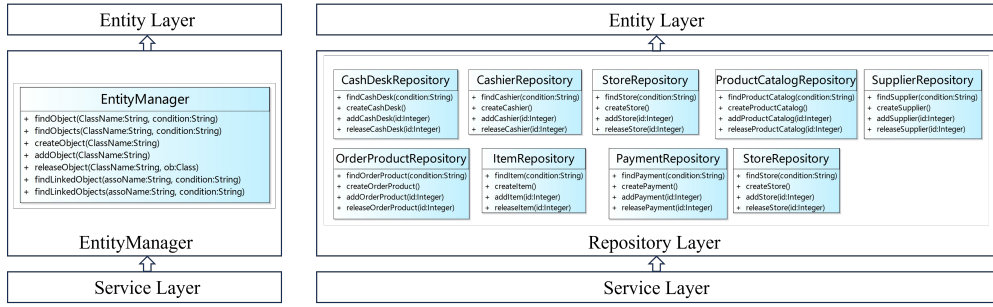
Table 2 Comparison of Design Quality Indicators Before and After Modification

Metric	DSC	NOH	ANA	DAM	DCC	CAM	MOA	MFA	NOP	CIS	NOM
Before	24.0000	3.0000	0.0833	1.0000	2.7917	1.0000	5.0000	0.4500	1.0000	144.0000	144.0000
After	32.0000	3.0000	0.0625	1.0000	2.0625	1.0000	5.0000	0.4500	1.0000	173.0000	173.0000
After/Before	1.3333	1.0000	0.7503	1.0000	0.7388	1.0000	1.0000	1.0000	1.0000	1.2014	1.2014

Table 3 Comparison of Design Quality Before and After Modification

Metric	Resuability	Flexibility	Understandability	Functionality	Extendibility	Effectiveness
Before	1.0000	1.0000	-0.9900	1.0000	1.0000	1.0000
After	1.3327	1.0653	-0.9979	1.1176	1.0058	0.9501

common interfaces. Less coupling between classes also leads to better reusability, flexibility, and extensibility. This example illustrates the effectiveness of the refactoring method according to quality metrics.

**Fig. 6** Optimized Class Diagram

4.3.2 LLM-based Design refactoring Agent

The rapid development of Large Language Models (LLMs), such as GPT, BERT, and their successors, has revolutionized various domains by providing unprecedented capabilities in natural language understanding and generation. These models are not only reshaping industries like customer support, content creation, and data analytics but are also increasingly being leveraged in specialized technical domains. In the realm of Object-Oriented (OO) design refactoring, LLMs have demonstrated their potential to analyze complex software models, identify design inefficiencies, and suggest improvements grounded in best practices and design principles such as SOLID and GRASP. By integrating domain-specific knowledge with pattern recognition, LLMs provide actionable recommendations, enhance maintainability, and reduce technical debt, making them a valuable tool for modern software engineering. In our approach, GPT-based capabilities are utilized in three key aspects:

- Providing design refactoring recommendations based on the current quality metrics.

Model refactoring Suggestion

PROMPT

The plantUML of Design is as follows: [DesignUML]. The design quality of this design are as follows: [DesignQuality].

Based on design patterns (e.g., Singleton, Factory, Observer, etc.) and GRASP design principles (e.g., Information Expert, Low Coupling, High Cohesion), suggest improvements for the model. Please explain the following:

- 1. Which classes or relationships can be optimized? Provide specific suggestions for changes.
- 2. Based on design patterns, which patterns could enhance the maintainability, scalability, or performance of the model?
- 3. How can GRASP principles optimize the assignment of responsibilities and interactions between classes?
- 4. How will the proposed changes address potential issues in the existing model (e.g., high coupling, low cohesion)?
- 5. After modifications, which aspects of the model (e.g., class responsibilities, relationships, coupling) will improve?

Please analyze each suggestion step by step (Chain of Thought), providing an explanation for each and proposing the most suitable solution.

The overall structure should be that the first part is a series of specific change suggestions, and the second part is explanations of the change suggestions.

VARIABLE

[DesignUML]: The plantUML representation of Design Model.

[DesignQuality]: Design quality assessment results automatically computed by the modeling tool, employing the QMOOD metrics, including reusability, flexibility, understandability, functionality, extendibility and effectiveness.

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

The provided prompt is designed to facilitate comprehensive and structured analysis of software design models, leveraging the capabilities of Large Language Models (LLMs) to optimize Object-Oriented (OO) design. It integrates principles from design patterns and GRASP (General Responsibility Assignment Software Patterns) to ensure that recommendations align with established best practices in software engineering. By using a PlantUML representation of the design model as input, the prompt focuses on actionable improvements, making it suitable for iterative development processes.

One of the key strengths of this prompt lies in its focus on clarity and step-by-step reasoning. The Chain of Thought (CoT) methodology is explicitly embedded, encouraging the model to break down complex design quality assessments into manageable components. This approach not only improves the interpretability of the recommendations but also ensures that the reasoning behind each suggestion is transparent and traceable. Furthermore, the prompt emphasizes a dual-structured

591 response, first presenting concrete recommendations and then providing detailed
592 justifications. This structure fosters a deeper understanding of how the proposed
593 changes will improve design quality metrics, such as maintainability, scalability, and
594 performance.

595 Another notable advantage is the explicit integration of both design patterns and
596 GRASP principles. By asking the model to evaluate the applicability of patterns
597 such as Singleton, Factory, and Observer, the prompt ensures that the recommenda-
598 tions are grounded in established strategies for solving common design challenges.
599 Simultaneously, GRASP principles guide the analysis of responsibility allocation
600 and interactions between classes, addressing issues like high coupling and low cohe-
601 sion. This dual focus enables a holistic quality assessment of the design, balancing
602 the structural and behavioral aspects of the model.

- 603 • Automatically generating explanations for changes in design quality metrics follow-
604 ing specific operations.

Model refactoring Explanation

PROMPT

The plantUML of Design is as follows: [DesignUML]. Based on the modified UML Class Diagram, explain why these changes will impact the quality of the model. Specifically, explain how the following aspects are improved or degraded:

- 1. Has the distribution of responsibilities among classes become more reasonable? Has cohesion been improved?
- 2. Has the coupling between classes been effectively reduced after the modifications?
- 3. How do the newly added design patterns or GRASP principles affect the scalability, maintainability, or performance of the model?
- 4. Has the design reduced complexity and improved the understandability of the model?
- 5. Overall, have the quality metrics of the model (e.g., flexibility, reusability) been improved? If so, in which specific areas?

Please explain step by step (Chain of Thought) the specific impact of each change.

VARIABLE

[DesignUML]: The plantUML representation of Design Model.

605

606 The accompanying prompt is crafted to enable detailed, step-by-step explanations
607 of the impacts of design modifications on software quality metrics. It provides
608 a structured framework that leverages the analytical capabilities of Large Lan-
609 guage Models (LLMs) to assess how specific changes in a UML Class Diagram
610 influence the overall design quality. By embedding Chain of Thought (CoT) rea-
611 soning, the prompt ensures that each modification is analyzed comprehensively and

612 methodically, making it a powerful tool for understanding and refining software
613 models.

614 A core strength of the prompt lies in its focus on critical aspects of design evaluation,
615 such as cohesion, coupling, scalability, and maintainability. These are foundational
616 metrics in assessing software quality, and the prompt explicitly calls for a systematic
617 analysis of how each metric is impacted by the proposed changes. For example, by
618 addressing whether responsibility distribution has become more logical or whether
619 cohesion has improved, the prompt ensures that the analysis is grounded in prin-
620 ciples of Object-Oriented Design (OOD). Similarly, the explicit focus on coupling
621 reduction and the application of design patterns or GRASP principles highlights
622 the importance of achieving balance in structural and behavioral design.

623 Moreover, the prompt emphasizes actionable insights by asking how design pat-
624 terns and GRASP principles enhance the model’s scalability, maintainability, and
625 performance. This ensures that the explanations are not only theoretical but also
626 practical, providing developers with a clear understanding of the trade-offs and ben-
627 efits introduced by specific design decisions. The inclusion of considerations like
628 complexity reduction and understandability further aligns the prompt with real-
629 world development goals, ensuring that the resulting models are both robust and
630 developer-friendly.

631 • Analyzing the traceability between the current design and specific requirements,
632 identifying unmet requirements, and suggesting complementary design elements to
633 address the gaps. The prompt is expertly designed to facilitate a comprehensive
634 and structured analysis of traceability between design and requirements in soft-
635 ware engineering. It leverages the capabilities of Large Language Models (LLMs) to
636 ensure precise mapping of requirements to design elements while identifying gaps
637 and proposing actionable solutions. The use of a PlantUML-based representation
638 for the design and a detailed requirements file ensures that the analysis is grounded
639 in clear, visual, and textual artifacts, enhancing the clarity and accuracy of the
640 recommendations.

641 A particularly valuable aspect of the prompt is its focus on unmet requirements.
642 By explicitly asking the model to identify and explain why certain requirements
643 are missing or overlooked, it fosters a deeper understanding of potential gaps in
644 the design process. This ensures that the analysis not only addresses technical
645 shortcomings but also considers contextual factors such as oversight, complexity, or
646 misalignment of goals.

647 The recommendation for complementary design elements further enhances the
648 prompt’s utility. By requiring the model to propose solutions aligned with estab-
649 lished best practices like SOLID principles, the prompt ensures that the suggestions
650 are both practical and theoretically sound. This dual focus on maintainability and
651 scalability reflects a strong alignment with modern software engineering priorities,
652 making the output highly actionable for practitioners.

653 The final step, justification and validation, adds an extra layer of rigor to the anal-
654 ysis. By asking the model to explain the reasoning behind proposed solutions and
655 discuss potential trade-offs, the prompt ensures that the recommendations are bal-
656 anced and well-considered. This not only aids in decision-making but also provides

657 valuable insights into the implications of the proposed changes, such as their impact
658 on design quality metrics like cohesion, coupling, and reusability.

Traceability Between Design and Requirements

PROMPT

The plantUML of Design is as follows: [DesignUML]. The Requirements is as follows: [Remodel].

I am designing a software system and need your expertise in analyzing the traceability between the current design and specific requirements. Please follow these steps to provide a detailed and actionable response:

- 1. Traceability Analysis: Begin by identifying the key requirements from the provided list. Map each requirement to the corresponding design elements.
- 2. Unmet Requirements Identification: Highlight any requirements that are not addressed by the current design. For each unmet requirement, explain why it might be missing or overlooked.
- 3. Recommendations for Complementary Design Elements: For each unmet requirement, propose specific design elements or modifications to the existing design. Ensure that the recommendations align with best practices in software design (e.g., SOLID principles, maintainability, scalability).
- 4. Justification and Validation: Provide reasoning for the proposed design elements, explaining how they fulfill the unmet requirements. Discuss any potential trade-offs or impacts on the overall design quality metrics.

Use a structured format for your response, such as:

Requirement X: (State the unmet requirement)

Current Coverage: (Briefly describe if it's partially covered or missing entirely)

Proposed Solution: (Detail the complementary design element)

Justification: (Explain why this solution is appropriate and its potential effects)

Finally, ensure that your response is concise yet comprehensive, avoiding unnecessary jargon. If additional clarification or details about the current design are needed, specify the information required.

VARIABLE

[plantUML]: The plantUML representation of Design Model.

[Remodel]: The Remodel file of Requirements.

659

660 **Discussion: Why OCL/Java Rather Than ATL or Epsilon.** A natural
661 question is whether a dedicated model transformation language such as ATL or
662 Epsilon would be a more appropriate vehicle for the transformation rules described
663 in this section. While ATL and Epsilon excel at expressing declarative, pattern-
664 matching mappings between metamodels, the rules in our approach involve two types

of logic that are more naturally expressed procedurally. First, the dependency analysis required to determine a valid execution order for OCL statements (Algorithm 2) constructs and traverses a dependency graph; this imperative, graph-centric computation does not map cleanly onto the declarative match-apply paradigm of ATL rules. Second, the sequence diagram generation rules must dynamically instantiate and link objects whose number and types are determined at runtime by the structure of each individual OCL contract, rather than by a fixed metamodel pattern. For these reasons, we implement the transformation logic in Java (integrated with the Sirius modeling workbench), using OCL as a *formal specification layer* to parse and interpret contract semantics. This hybrid strategy retains the readability and formal grounding of OCL for contract analysis while leveraging the full expressiveness of Java for the procedural coordination logic. We acknowledge that this places an additional burden on end users who must author OCL contracts; tooling support for contract authoring and validation (already partially provided by the RM2PT framework) is an avenue for future improvement.

5 Evaluation

In this section, We use six requirement models of different software systems to perform evaluation.

5.1 Research Questions

- **RQ1:** To what extent do the transformation rules proposed by our approach cover various situations?
- **RQ2:** How is the quality of the generated design models, and how does it compare to manually created models?
- **RQ3:** Does our automated approach for constructing design models offer a time efficiency advantage over manual approaches?.

5.2 Experimental Setting

We developed a tool based on the **Eclipse Modeling framework** to automate our approach. We use ATL transformation language to implement the requirements model to design model transformation and Sirius to graphically display and modify the design model. The tool provides the functionality to edit the generated design models. During the design model generation and modification process, automatic metrics calculation and identification of design elements to be optimized are achieved by writing Java Services for Sirius.

To address RQ1, we start by counting the relevant measures within both the requirement model and the automatically generated design model. These counts are then used to calculate specific metrics that are crucial for our analysis.

For RQ2, we enlist the expertise of seasoned professionals in the software engineering field. These experts are carefully recruited based on their extensive experience and in-depth knowledge of design models and domain modeling. They are tasked with establishing design models that conform to an anaemic domain model, derived directly

from the requirement models. These expert-generated models serve as the reference models for our study. To facilitate comparison, we also involve postgraduate students who are specializing in software engineering. These students are recruited through academic channels, focusing on those who have demonstrated proficiency in software design and development in their coursework.

Before the postgraduates begin constructing the design models, they undergo a comprehensive training program. This program is structured to ensure they have a thorough understanding of the tools and methodologies they will be using, as well as the concepts of requirement modeling and anaemic domain models. The training involves hands-on sessions where the participants are introduced to the tools necessary for constructing design models. They also receive detailed explanations of the models they are expected to build, focusing particularly on the principles of the anaemic domain model and how it aligns with the requirement model.

Once the training is complete, the participants are provided with a specific set of requirement documents. Their task is to construct a design model based on these documents, adhering to the principles of the anaemic domain model. The design model they are asked to create is for a monolithic system built using SpringBoot. This part of the experiment is critical as it allows us to directly compare the manually constructed design models with those generated automatically, thereby providing insights into the effectiveness and efficiency of each approach.

For RQ3, we focus on the time efficiency of the design model construction process. We meticulously record the time taken by the postgraduates to manually construct the design model. This duration is then compared with the time required to build the model using our automated method. The comparison of these time metrics provides valuable data on the efficiency gains that our automated approach offers over traditional manual methods.

5.3 Case Studies

The dataset has six requirement models, four of which come from RM2PT. The cases of the six requirement models describe the requirements of six common scenarios, including automatic teller machine(ATM), supermarket system(CoCoME), library management system(LibMS), loan management system(LoanPS), Parking spot share system(ParkShare) and Airport management system(AirMS). The case of an automatic teller machine mainly involves the user's need to use an ATM to deposit and withdraw money. It is relatively simple and can be used as a rapid preliminary verification of the automatic generation method of the design model. The case of CoCoME describes the needs of the cashier in the supermarket to process the shopping documents and the administrator to manage the supermarket inventory. The process involved in this case is complex, and it can be verified whether the generated model can describe the interaction between the objects in the system well. The case of LibMS describes the need for students to borrow and return books. In this case, the system operation is complex, which can verify whether the generated design model can encapsulate the system operation well. The case of LoanPS describes the user's demand for loans. To fulfill this demand, it is necessary to call the services of the third party, which can verify whether the generated design model can well describe the scenario where the

system interacts with other systems to complete functions. The ParkShare case allows people to rent or share their available parking spaces, which are relatively small in scale. The AirMS case addresses the needs of airport equipment management and has a larger scale. Both case studies are derived from real systems and have requirement models constructed by experienced developers. The complexity of the six requirement models is close to the complexity of the enterprise monolithic system that this study is aimed at, and it is suitable for the data set of this experiment. Make statistics on the number of model elements in the requirement model, and the statistical results are shown in Table 4.

Table 4 The Complexity of Requirements Models

Case Study	Actor	Use Case	SO	OCL	Entity Class	Association
ATM	2	6	15	65	3	4
CoCoME	3	16	43	198	13	20
LibMS	7	19	45	303	11	17
LoanPS	5	10	34	127	12	8
ParkShare	4	7	12	48	7	11
AirMS	6	33	58	336	15	19
Sum	27	91	207	1077	61	79

* SO is the abbreviations of system operations. OCL refers to the number of OCL statements that can be converted into messages in the sequence diagram.

5.4 Evaluation Metrics

5.4.1 Transformation Rule Metrics

These metrics evaluate the extent to which the proposed transformation rules cover the transformation from requirements models to design models.

- **Success rate of entity-to-class transformation.** $CSRate = RC/AC$, where RC denotes the number of correct entity classes in the class diagram, and AC denotes the number of entities in the conceptual class diagram.
- **Success rate of association-to-relationship transformation.** $ASRate = RA/AA$, where RA denotes the number of correct relationships between entity classes in the class diagram, and AA denotes the number of associations in the conceptual class diagram.
- **Correctness of encapsulation of operations.** $OSRate = RO/AO$, where RO indicates the number of system operations in the class diagram that are correctly encapsulated into interface classes, and AO indicates the total number of system operations in contracts.
- **Success rate of message generation in sequence diagram.** $MSRate = RM/AM$, where AM refers to the number of OCL expressions to be converted, and RM refers to the number of messages correctly converted from OCL expressions.

776 Similarly, we can calculate $Def - MSRate$ and $Post - MSRate$ for the contract's
 777 definition and postcondition parts, respectively.

778 5.4.2 Design Quality Metrics

779 We evaluate design quality using high-level quality attribute metrics in QMOOD. We
 780 take the quality attributes of design models manually constructed by experts as the
 781 basis and normalize the quality attribute indicators of automatic and semi-automatic
 782 generated design models with a few fast iterations for comparison.

783 For the sake of descriptive clarity, formulas for calculating the high-level design
 784 metrics in QMOOD are provided.

$$Reusability = -0.25 * DCC + 0.25 * CAM + 0.5 * CIS + 0.5 * DSC$$

785

$$Flexibility = 0.25 * DAM - 0.25 * DCC + 0.5 * MOA + 0.5 * NOP$$

786

$$Understandability = -0.33 * ANA + 0.33 * DAM - 0.33 * DCC \\ + 0.33 * CAM - 0.33 * NOP - 0.33 * NOM - 0.33 * DSC$$

$$Functionality = 0.12 * CAM + 0.22 * NOP + 0.22 * CIS + 0.22 * DSC + 0.22 * NOH$$

787

$$Extendibility = 0.5 * ANA - 0.5 * DCC + 0.5 * MFA + 0.5 * NOP$$

788

$$Effectiveness = 0.2 * ANA + 0.2 * DAM + 0.2 * MOA + 0.2 * MFA + 0.2 * NOP$$

789 The meanings of the variables on the right-hand side of the equation are shown in the
 790 following Table 5.

791 5.5 Evaluation Results

792 5.5.1 RQ1: To what extent do the transformation rules proposed 793 by our approach cover various situations

794 Our tool can successfully and correctly generate all class diagrams of these six case
 795 studies. The results of the sequence diagram generation are shown in Table 6. On
 796 average, 98.50% (DSRate) of the expressions in the definition part of the contract
 797 is correctly transformed into messages in the sequence diagram, 93.96% (PSRate) in
 798 the post-condition part, and 94.80% (MSRate) of the overall messages are correctly
 799 generated. This result shows that our proposed rule can completely cover the case

Table 5 Meaning of Variables

Variables	Meaning
DSC	Design Size in Classes
NOH	Number of Hierarchies
ANA	Average Number of Ancestors
DAM	Data Access Metric
MFM	Measure of Functional Modularity
DCC	Direct Class Coupling
CAM	Cohesion Among Methods in Class
MOA	Measure of Aggregation
MFA	Measure of Function Abstraction
NOP	Number of Polymorphic Methods
CIS	Class Interface Size
NOM	Number of Methods

Table 6 The Generation Result of Sequence Diagram Message

Case Study	Def-OCL	Def-Message	DSRate	Post-OCL	Post-Message	PSRate	MSRate
ATM	9	9	100.00%	56	48	85.71%	87.69%
CoCoME	35	34	97.14%	163	156	95.71%	95.71%
LibMS	59	57	96.61%	244	223	91.39%	92.41%
LoanPS	19	19	100.00%	108	104	96.30%	96.85%
ParkShare	5	5	100.00%	43	42	97.67%	97.92%
AirMS	73	73	100.00 %	263	251	95.44%	96.43%
Sum	200	197	98.50%	877	824	93.96%	94.80%

* Def-OCL refers to the OCL expressions to be converted in the definition part of the system operation contract, and Def-Message refers to the number of messages correctly converted from Def-OCL; Post-OCL refers to the OCL expressions to be converted in the post-condition part of the system operation contract, and Post-Message refers to the number of messages correctly converted from Post-OCL.

of class diagram generation and can basically cover the case of sequence diagram generation.

After analysis, the failure to generate messages includes the following:

1. The developed tool cannot handle multiple nested If-else statements.
2. The contract statement is wrongly written.
3. The return value of the contract is not a variable, but an OCL operation statement.

5.5.2 RQ2: How is the quality of the generated design models, and how does it compare to manually created models

In our study, we used the design model that conforms to the anemic domain model constructed by experts as the correct reference model. Based on this reference model, we calculated the model quality metrics for design models manually constructed by graduate students in the software engineering field and those automatically generated by our tool, as shown in Table 7. The arithmetic mean calculation based on six experimental datasets reveals that design models generated by RapidOOD achieve 89.20% (without human intervention) and 94.56% (under human-in-the-loop collaboration) of the quality assessment metrics attained by expert-developed models, respectively.

Table 7 Comparison of Results: Pure MDE, LLM-Only, and RapidOOD

Case Study	Quality	LLM Only		Pure MDE	RapidOOD(LLM+Rule)	
		Single LLM	Multi LLM	Rule Only	Without Human	Human in the Loop
ATM	Reusability	0.49	0.90	0.58	0.87	1
	Flexibility	0.15	0.24	0.43	0.8	1
	Understandability	-0.21	-1.16	-0.56	-0.93	-1
	Funcationality	0.44	0.49	0.65	0.98	0.98
	Extendibility	0.50	0.42	0.36	1	1
	Effectiveness	0.32	0.34	0.80	0.99	0.99
Case Study	Quality	Single LLM	Multi LLM	Rule Only	Without Human	Human in the Loop
CoCoME	Reusability	0.38	0.47	0.68	0.86	1
	Flexibility	0.18	0.23	0.08	0.98	1
	Understandability	-0.02	-0.15	-0.67	-0.98	-1
	Funcationality	0.37	0.29	0.69	0.96	0.96
	Extendibility	0.25	0.47	0.45	0.86	1
	Effectiveness	0.39	0.37	0.26	0.93	0.98
Case Study	Quality	Single LLM	Multi LLM	Rule Only	Without Human	Human in the Loop
LibraryMS	Reusability	0.34	0.42	0.56	0.90	1
	Flexibility	0.32	0.37	0.77	0.98	1
	Understandability	-0.24	-0.12	-0.56	-0.97	-1
	Funcationality	0.35	0.30	0.59	0.99	0.99
	Extendibility	0.32	0.33	0.36	0.89	0.97
	Effectiveness	0.36	0.43	0.60	0.96	0.98
Case Study	Quality	Single LLM	Multi LLM	Rule Only	Without Human	Human in the Loop
LoanPS	Reusability	0.51	0.56	0.16	0.89	0.98
	Flexibility	0.21	0.08	0.36	0.96	1
	Understandability	-0.42	-0.52	-0.17	-0.98	-1
	Funcationality	0.39	0.48	0.23	0.99	0.99
	Extendibility	0.42	0.48	0.32	0.94	1
	Effectiveness	0.34	0.26	0.40	0.97	1
Case Study	Quality	Single LLM	Multi LLM	Rule Only	Without Human	Human in the Loop
ParkShare	Reusability	0.32	0.66	0.35	0.96	1
	Flexibility	0.01	0.14	0.54	0.92	1
	Understandability	-0.05	-0.21	-0.35	-0.97	-1
	Funcationality	0.22	0.38	0.42	0.98	0.98
	Extendibility	0.50	0.48	0.32	0.93	1
	Effectiveness	0.20	0.30	0.57	0.97	0.99
Case Study	Quality	Single LLM	Multi LLM	Rule Only	Without Human	Human in the Loop
AirMS	Reusability	0.55	0.28	0.19	0.87	0.99
	Flexibility	0.23	0.20	0.24	0.90	0.97
	Understandability	-0.32	-0.17	-0.20	-0.93	-0.99
	Funcationality	0.64	0.36	0.24	0.93	0.96
	Extendibility	0.39	0.44	0.44	0.95	0.96
	Effectiveness	0.33	0.34	0.50	0.95	0.96

¹ We adopt the design model manually constructed by experts based on the requirements model as the benchmark. The results in the table are the ratios of quality attributes compared to the benchmark.

² *LLM Only* column: results obtained solely by posing questions to the LLM. *Single LLM*: Requirements model provided to LLM for direct design model generation. *Multi LLM*: Design model from 5 "Generate - Evaluate - Optimize" iteration with 3 LLM nodes.

³ *Pure MDE* column: results obtained by rule-based design generation and rule-based refactoring only, without LLM involvement.

⁴ *RapidOOD* column: *Without Human* - initial design by Design Generation Agent only; *Human in the Loop* - initial design adjusted by human per evaluation and refactoring suggestions after a few iterations.

In all LLM-involved experiments, we used **GPT-4o** (**gpt-4o**) as the underlying language model, which is hardcoded in the tool implementation. Each LLM call follows a two-part prompt structure: a *format-constrained System Prompt* paired with a *structured User Prompt*. The System Prompt is responsible for enforcing output format constraints (e.g., requiring the model to respond exclusively in well-formed JSON or UML-compatible notation), while the User Prompt supplies the contextual input data—including class diagrams, OCL contracts, and few-shot examples—together with an explicit specification of the expected output format. This separation of concerns

ensures that format compliance is handled at the system level, keeping User Prompts focused on domain-specific content.

We also compared the performance of our method with that of directly using large language models (LLMs), including single LLM and multi LLMs. For single LLMs, we provided the requirement model to the LLM and requested it to directly generate the corresponding design model. As for multi LLMs, we created a workflow that includes three LLM nodes. One was responsible for design generation, another for quality assessment, and the third for refactoring. The process of generation-assessment-refactoring was iterated five times, after which the final result was output.

In addition, we compared our method against *Pure MDE*, which performs rule-based design generation and rule-based refactoring exclusively, without any LLM involvement. This baseline represents a traditional model-driven engineering pipeline where transformation rules fully determine the output. As shown in Table 7, Pure MDE achieves an average quality score of approximately 43.5% relative to the expert-constructed reference models across the six case studies. Although this result surpasses the Single LLM approach (approximately 32.5% on average), it falls substantially short of RapidOOD, which achieves 89.20% (without human intervention) and 94.56% (human-in-the-loop).

The comparison protocol therefore includes five groups: Single LLM, Multi-LLM, Pure MDE/Rule Only, RapidOOD without Human, and RapidOOD with Human. The first two groups test LLM-only generation and iterative LLM generation, respectively; Pure MDE tests deterministic model transformation; and the two RapidOOD groups test the contribution of the integrated rule-LLM workflow and human collaboration.

According to the findings, we can conclude that:

1. The initial designs generated by our method can achieve a quality level close to that of experts. Moreover, with the assistance of software, after a few manual adjustments, the designs can reach the level of experts in some indicators, and overall, attain the same quality standard as that of experts.
2. Compared with methods that merely rely on LLMs, our approach exhibits substantial advantages in quality metrics. Through analysis, the primary reason is that the scale of results output by LLMs is limited, and this limited scale confers better understandability. In terms of indicators such as Reusability, Flexibility, Functionality, Extendibility, and Effectiveness, methods solely using LLMs can hardly reach a level comparable to ours. This reflects the benefits of integrating the model-driven approach with LLMs. Employing a rule-based method to generate the initial design can: 1) Ensure the initial scale of the design model; 2) Guarantee the traceability between the design model and the requirement model; 3) Prevent the introduction of hallucinations from the output of LLMs.
3. Compared with the Pure MDE baseline, RapidOOD achieves substantially higher quality scores across the six case studies. While Pure MDE guarantees structural completeness and full traceability through deterministic rule application, it lacks the capacity for semantic-level optimization and context-aware design decisions.

5.5.3 RQ3: Does our automated approach for constructing design models offer a time efficiency advantage over manual approaches

In the process of manually constructing the design model, we provided the requirement model, which typically includes use case diagrams, class diagrams, and system sequence diagrams. In our approach, we additionally employed OCL contracts to constrain system operations. The time spent on writing OCL contracts constitutes the primary time consumption of our automated method. Therefore, we compared the time consumed by OCL contracts with that of manually constructing design models, as shown in Table 8. As can be seen, the average time taken to construct OCL contracts is 4.6 hours, while the average time for manually constructing design models is 6.4 hours. Our approach can save 28.1% of the time consumption, achieving 1.39 times the efficiency of the manual method.

Table 8 Time Consuming of Automated Mean and Manual Mean

Case Study	OCL Cost(h)	Manual Mean(h)	Time Saving Rate
ATM	1.3	2.0	35.00%
CoCoME	4.9	7.5	34.67%
LibMS	6.4	8.3	22.89%
LoanPS	6.9	7.2	4.17%
ParkShare	3.1	4.2	26.19%
AirMS	5.0	9.2	45.65%
Average	4.6	6.4	28.09%

5.6 Discussion and Limitation

Using lightweight formal requirement models as input, our approach can successfully generate design models that conform to the anemic domain model. Our approach outperforms graduate students in the software engineering field in terms of completeness and consistency with expert-constructed design models. However, our method has the following limitations.

Our approach does not currently support the handling of multiple nested logical judgments. This limitation restricts the ability of our tool to capture complex relationships and constraints within the requirements. Nested logical judgments often occur in real-world systems, especially in those with intricate business rules. Therefore, extending our approach to support such constructs would significantly enhance its applicability and robustness.

Our method relies on users to correctly write OCL (Object Constraint Language) contracts. While OCL provides a powerful means to express constraints, it is a specialized language that may not be familiar to all stakeholders involved in the software development process. Thus, the need for expertise in OCL might limit the usability of our approach, particularly for non-technical stakeholders. Providing better support for

OCL validation or integrating a user-friendly interface for generating OCL contracts could help mitigate this issue.

Currently, our approach only generates anemic domain model designs and cannot support the generation of a wider variety of design models, such as rich domain models or domain-driven design patterns. Anemic domain models, while useful, do not always capture the full richness of the domain logic. Expanding our approach to support other design patterns could improve the expressiveness and flexibility of the generated design models.

5.7 Threats to Validity

There are several threats to the validity of our study.

The dataset used in this experiment consists of only six requirement models, limiting the generalizability of our findings. The automatic generation of the class diagram in the design model is only conducted six times, and the evaluation of the success rate of the automatic generation of the class diagram may not be sufficient to fully validate the effectiveness of our approach across different domains and complexities.

The complexity of the data centralized system used in this experiment is close to that of a single enterprise information system. However, there are still differences in the actual business processes and requirements, which may affect the rationality of the experimental evaluation. To address this, future work should include experiments with a broader range of systems and varying levels of complexity.

We use expert-constructed design models as the correct answers for comparison. While this approach is common, it may introduce human bias. Expert opinions can vary, and there may not be a single correct design solution. To mitigate this threat, we could conduct additional studies involving multiple experts and consensus-building processes to ensure the reliability of the baseline models.

6 Conclusion and Future Work

This paper proposes the RapidOOD approach to automated design model generation from lightweight formal requirements models. Work like RM2PT can help achieve a validated requirements model by automatically generating prototypes from requirements models. RapidOOD generates OO design models from this validated requirements model, further alleviating the software development problem. The generated design models can be used as references for designers and shorten the time-consuming design phase. Six case studies evaluated the capabilities of RapidOOD. The experiment result is satisfactory and shows that the approach is effective and efficient.

In the future, We will consider combining the anemic model and domain model design philosophies to guide the automated generation of designs, thus addressing the automatic generation of design models for a broader range of systems.

Acknowledgements. This work was supported by the National Natural Science Foundation of China (No.62302029), State Key Laboratory of Complex & Critical Software Environment.

References

- [1] Eden, A.H., Kazman, R.: Architecture, design, implementation. In: 25th International Conference on Software Engineering, 2003. Proceedings., pp. 149–159 (2003). <https://doi.org/10.1109/ICSE.2003.1201196>
- [2] Flageol, W., Menaud, É., Guéhéneuc, Y.-G., Badri, M., Monnier, S.: A mapping study of language features improving object-oriented design patterns **160**, 107222 <https://doi.org/10.1016/j.infsof.2023.107222> . Accessed 2024-04-01
- [3] Li, X., Liu, Z., He, J.: Formal and use-case driven requirement analysis in uml. In: 25th Annual International Computer Software and Applications Conference. COMPSAC 2001, pp. 215–224 (2001). <https://doi.org/10.1109/CMPSAC.2001.960619>
- [4] Liu, Z., Jifeng, H., Li, X., Chen, Y.: A relational model for formal object-oriented requirement analysis in uml. In: Dong, J.S., Woodcock, J. (eds.) Formal Methods and Software Engineering, pp. 641–664. Springer, Berlin, Heidelberg (2003)
- [5] Chen, Z., Liu, Z., Ravn, A.P., Stolz, V., Zhan, N.: Refinement and verification in component-based model-driven design. Science of Computer Programming **74**(4), 168–196 (2009) <https://doi.org/10.1016/j.scico.2008.08.003> . Special Issue on the Grand Challenge
- [6] Yang, Y., Li, X., Liu, Z., Ke, W.: Rm2pt: A tool for automated prototype generation from requirements model. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), pp. 59–62 (2019). <https://doi.org/10.1109/ICSE-Companion.2019.00038>
- [7] Yang, Y., Ke, W., Li, X.: Rm2pt: Requirements validation through automatic prototyping. In: 2019 IEEE 27th International Requirements Engineering Conference (RE), pp. 484–485 (2019). <https://doi.org/10.1109/RE.2019.00067>
- [8] Yang, Y., Li, X., Ke, W., Liu, Z.: Automated prototype generation from formal requirements model. IEEE Transactions on Reliability **69**(2), 632–656 (2020) <https://doi.org/10.1109/TR.2019.2934348>
- [9] Bao, T., Yang, J., Yang, Y., Yin, Y.: Rm2doc: a tool for automatic generation of requirements documents from requirements models. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. ICSE '22, pp. 188–192. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3510454.3516850>
- [10] Easterbrook, S., Lutz, R., Covington, R., Kelly, J., Ampo, Y., Hamilton, D.: Experiences using lightweight formal methods for requirements modeling. IEEE Transactions on Software Engineering **24**(1), 4–14 (1998) <https://doi.org/10.1109/32.663994>

- 973 [11] Mala, G.S.A., Uma, G.V.: Automatic construction of object oriented design mod-
 974 els [uml diagrams] from natural language requirements specification. In: Yang, Q.,
 975 Webb, G. (eds.) PRICAI 2006: Trends in Artificial Intelligence, pp. 1155–1159.
 976 Springer, Berlin, Heidelberg (2006)
- 977 [12] Sharma, V.S., Sarkar, S., Verma, K., Panayappan, A., Kass, A.: Extracting high-
 978 level functional design from software requirements. In: 2009 16th Asia-Pacific
 979 Software Engineering Conference, pp. 35–42 (2009). [https://doi.org/10.1109/](https://doi.org/10.1109/APSEC.2009.63)
 980 [APSEC.2009.63](https://doi.org/10.1109/APSEC.2009.63)
- 981 [13] Yue, T., Briand, L.C., Labiche, Y.: atoucan: An automated framework to derive
 982 UML analysis models from use case models. *ACM Trans. Softw. Eng. Methodol.*
 983 **24**(3), 13–11352 (2015)
- 984 [14] Easterbrook, S., Lutz, R., Covington, R., Kelly, J., Ampo, Y., Hamilton, D.:
 985 Experiences using lightweight formal methods for requirements modeling. *IEEE*
 986 *Transactions on Software Engineering* **24**(1), 4–14 (1998) [https://doi.org/10.](https://doi.org/10.1109/32.663994)
 987 [1109/32.663994](https://doi.org/10.1109/32.663994)
- 988 [15] Li, X., Liu, Z., He, J.: Formal and use-case driven requirement analysis in uml.
 989 In: 25th Annual International Computer Software and Applications Conference.
 990 COMPSAC 2001, pp. 215–224 (2001). [https://doi.org/10.1109/CMPSAC.2001.](https://doi.org/10.1109/CMPSAC.2001.960619)
 991 [960619](https://doi.org/10.1109/CMPSAC.2001.960619)
- 992 [16] Chen, Z., Hannousse, A.H., Van Hung, D., Knoll, I., Li, X., Liu, Z., Liu, Y.,
 993 Nan, Q., Okika, J.C., Ravn, A.P., Stolz, V., Yang, L., Zhan, N.: In: Rausch, A.,
 994 Reussner, R., Mirandola, R., Plášil, F. (eds.) *Modelling with Relational Calculus of Object and Component Systems - rCOS*, pp. 116–145. Springer, Berlin,
 995 Heidelberg (2008). https://doi.org/10.1007/978-3-540-85289-6_6
- 997 [17] Chen, Z., Liu, Z., Ravn, A.P., Stolz, V., Zhan, N.: Refinement and verification in
 998 component-based model-driven design. *Science of Computer Programming* **74**(4),
 999 168–196 (2009) <https://doi.org/10.1016/j.scico.2008.08.003> . Special Issue on the
 1000 Grand Challenge
- 1001 [18] Liu, Z., Morisset, C., Stolz, V.: rcos: Theory and tool for component-based model
 1002 driven development. In: Arbab, F., Sirjani, M. (eds.) *Fundamentals of Software*
 1003 *Engineering*, pp. 62–80. Springer, Berlin, Heidelberg (2010)
- 1004 [19] Koch, N., Zhang, G., Escalona, M.J.: Model transformations from requirements
 1005 to web system design. In: *Proceedings of the 6th International Conference on*
 1006 *Web Engineering. ICWE '06*, pp. 281–288. Association for Computing Machin-
 1007 *ery*, New York, NY, USA (2006). <https://doi.org/10.1145/1145581.1145637> .
 1008 <https://doi.org/10.1145/1145581.1145637>
- 1009 [20] Mussbacher, G., Kienzle, J., Amyot, D.: Transformation of aspect-oriented

- requirements specifications for reactive systems into aspect-oriented design specifications. In: 2011 Model-Driven Requirements Engineering Workshop, pp. 39–47 (2011). <https://doi.org/10.1109/MoDRE.2011.6045365>
- [21] Chidamber, S.R., Kemerer, C.F.: A metrics suite for object oriented design. IEEE Transactions on Software Engineering **20**(6), 476–493 (1994) <https://doi.org/10.1109/32.295895>
- [22] Lorenz, M., Kidd, J.: Object-oriented Software Metrics: a Practical Guide. Prentice-Hall, Inc., USA (1994)
- [23] Abreu, F.B., Carapuça, R.: Object-oriented software engineering: Measuring and controlling the development process. In: Proceedings of the 4th International Conference on Software Quality, vol. 186, pp. 1–8 (1994)
- [24] Bansiya, J., Davis, C.G.: A hierarchical model for object-oriented design quality assessment. IEEE Transactions on Software Engineering **28**(1), 4–17 (2002) <https://doi.org/10.1109/32.979986>
- [25] Pham, V., Lokan, C., Kasmarik, K.: A better set of object-oriented design metrics for within-project defect prediction. In: Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering. EASE '20, pp. 230–239. Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3383219.3383243> . <https://doi.org/10.1145/3383219.3383243>
- [26] Yan, M., Xia, X., Zhang, X., Xu, L., Yang, D.: A systematic mapping study of quality assessment models for software products. In: 2017 International Conference on Software Analysis, Testing and Evolution (SATE), pp. 63–71 (2017). <https://doi.org/10.1109/SATE.2017.16>
- [27] Insfran, E., Gonzalez-Huerta, J., Abrahão, S.: Design guidelines for the development of quality-driven model transformations. In: Petriu, D.C., Rouquette, N., Haugen, Ø. (eds.) Model Driven Engineering Languages and Systems, pp. 288–302. Springer, Berlin, Heidelberg (2010)
- [28] Rähä, O., Koskimies, K., Mäkinen, E.: Generating software architecture spectrum with multi-objective genetic algorithms. In: 2011 Third World Congress on Nature and Biologically Inspired Computing, pp. 29–36 (2011). <https://doi.org/10.1109/NaBIC.2011.6089413>
- [29] Bowman, M., Briand, L.C., Labiche, Y.: Solving the class responsibility assignment problem in object-oriented analysis with multi-objective genetic algorithms. IEEE Transactions on Software Engineering **36**(6), 817–837 (2010) <https://doi.org/10.1109/TSE.2010.70>
- [30] Hadaytullah, Vathsavayi, S., Rähä, O., Koskimies, K.: Tool support for software

- 1047 architecture design with genetic algorithms. In: 2010 Fifth International Confer-
1048 ence on Software Engineering Advances, pp. 359–366 (2010). [https://doi.org/10.](https://doi.org/10.1109/ICSEA.2010.61)
1049 [1109/ICSEA.2010.61](https://doi.org/10.1109/ICSEA.2010.61)
- 1050 [31] Tian, Z., Yang, Y., Cheng, S.: Rm2dm: A tool for automatic generation of
1051 oo design models from requirements models. In: 2023 IEEE/ACM 45th Inter-
1052 national Conference on Software Engineering: Companion Proceedings (ICSE-
1053 Companion), pp. 36–40 (2023). [https://doi.org/10.1109/ICSE-Companion58688.](https://doi.org/10.1109/ICSE-Companion58688.2023.00020)
1054 [2023.00020](https://doi.org/10.1109/ICSE-Companion58688.2023.00020)
- 1055 [32] Vernon, V.: Implementing Domain-driven Design, pp. 1–20. Addison-Wesley, ???
1056 (2013)
- 1057 [33] Wooldridge, M., Ciancarini, P.: Agent-oriented software engineering: The state
1058 of the art. In: Ciancarini, P., Wooldridge, M.J. (eds.) Agent-Oriented Software
1059 Engineering, pp. 1–28. Springer, Berlin, Heidelberg (2001)